

NOT ALL ERRORS MATTER: SEVERITY-AWARE SOFT ERROR DETECTION FOR VISION LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Vision-Language Models (VLMs) are increasingly deployed in safety-critical applications such as autonomous driving and remote sensing, yet their inference remains vulnerable to Silent Data Corruption (SDC) caused by GPU soft errors. We observe that the majority of hardware faults are naturally masked by the model, while a small fraction induces significant semantic deviations. Moreover, the magnitude of numerical anomalies in activations does not strictly correlate with the degree of semantic corruption in outputs, rendering conventional numerical-threshold approaches inadequate—they struggle to detect critical faults reliably while incurring substantial unnecessary detection overhead. To address this, we propose SIEVE, a lightweight, semantic-severity-aware soft-error detection framework for VLMs. SIEVE orthogonally projects the self-attention outputs of adjacent decoder layers into a low-dimensional space and trains layer-aware predictors on clean execution traces to model the inter-layer mapping. At inference time, the deviation of actual outputs from the predicted mapping is aggregated into a Discrepancy Profile, which is fed to a lightweight XGBoost classifier to identify Significant SDC events. Across three VLMs and three VQA benchmarks, SIEVE achieves 96.38% precision, 91.49% recall, and 93.78% F1 on the primary evaluation set, with a false positive rate (FPR) of only 0.107%. Compared to Ranger and Dr.DNA under identical configurations, SIEVE improves F1 by 10.33 and 11.38 percentage points, respectively, while introducing an average end-to-end overhead of only 6.65%.

1 INTRODUCTION

In recent years, Vision-Language Models (VLMs), with their multimodal perception and reasoning capabilities, have been increasingly deployed in safety-critical domains such as autonomous driving Sima et al. (2024); Tian et al. (2024); Wei et al. (2025), remote sensing analysis Wang et al. (2025), and medical diagnosis Alsabbagh et al. (2025). These models typically rely on large-scale GPUs or other AI accelerators, whose execution is susceptible to transient hardware faults induced by cosmic rays, voltage fluctuations, and device aging Wang et al. (2023); He et al. (2023); Tan et al. (2025). Such faults can silently alter computational results without triggering any system exception, forming Silent Data Corruption (SDC). Public reports highlight this risk: Meta encountered six SDC-induced training interruptions within 54 days during the large-scale training of LLaMA 3, and Google reported SDC events disrupting Gemini training on TPUv5 clusters roughly every one to two weeks. SDC has thus become a non-negligible reliability risk in large-scale AI infrastructure; for safety-critical VLM inference, an undetected SDC may further translate into erroneous semantic outputs and decisions.

Through extensive fault injection experiments, we observe that SDCs differ markedly in their impact on VLM outputs: as shown in Figure 1a, a fault may cause a large numerical change while the faulty answer remains semantically consistent with the fault-free one; Figure 1b shows a case where both the numerical and semantic deviations are small, with no visible effect on the result; in contrast, Figures 1c and d show that even a small numerical perturbation can lead to missing key facts, misidentified objects, or reversed conclusions. We define SDCs that induce such significant semantic deviations as **Significant SDCs**; in safety-critical scenarios such as autonomous driving, this class of errors may cause severe consequences and is therefore the detection target of this work.

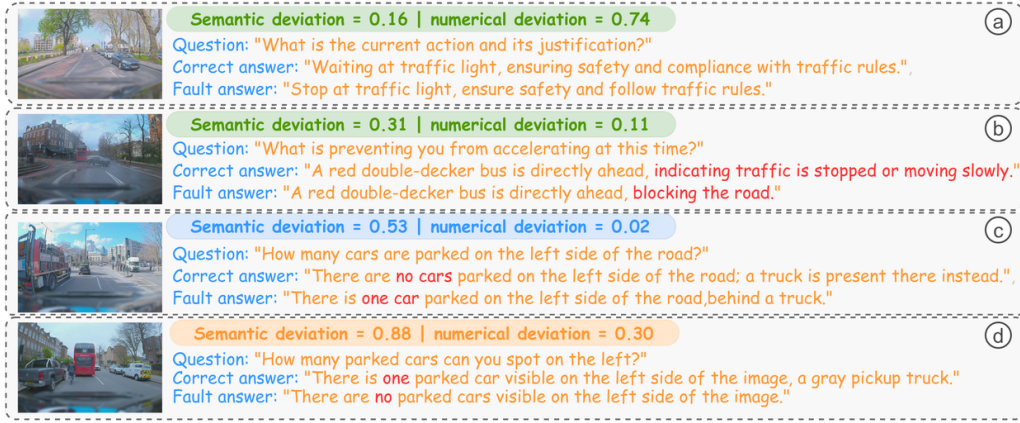


Figure 1: Examples of SDC outputs with different numerical and semantic deviations.

Existing soft error detection methods typically assess fault impact based on the activation range or numerical distribution. However, in VLMs there is no stable one-to-one correspondence between the magnitude of numerical anomalies and the semantic damage to the output: a large numerical perturbation may be absorbed by subsequent computation, while a small one may propagate through the network and alter key generated content. Fixed numerical thresholds therefore tend to suffer from both missed Significant SDCs and false alarms on non-critical faults. An effective detector must further characterize how faults affect the transformation of the model’s internal semantic representations.

Guided by this observation, we propose **SIEVE**, a lightweight, semantic-severity-aware soft error detection framework for VLMs. SIEVE monitors selected adjacent decoder layers during inference, projects their intermediate representations into a low-dimensional space, and uses a layer-aware predictor trained on clean execution traces to model the nominal inter-layer mapping. At runtime, the deviation of the actual representations from this mapping is aggregated into a layer-wise **Discrepancy Profile**, which a lightweight classifier calibrated independently judges whether the current execution suffers a Significant SDC. The method requires no online access to ground-truth answers and modifies no VLM parameters, serving as a front-end detection module for subsequent error-recovery mechanisms.

We systematically evaluate SIEVE on three representative VLMs and three VQA datasets. Under an evaluation protocol with semantically isolated splits and frozen thresholds (§4), SIEVE achieves a macro-average F1 of 93.78% and a non-Significant-SDC false positive rate of 0.107% on the Final Test of the nine tasks, with an end-to-end overhead of about 6.65%, substantially outperforming the Ranger and Dr.DNA baselines under identical configurations.

Our main contributions are summarized as follows:

- We formulate the Significant SDC problem for VLM inference reliability, using an LLM-as-a-Judge with a fixed rubric to distinguish slight from significant semantic deviations, and reveal through systematic fault injection the inconsistency between the magnitude of numerical anomalies and the final semantic damage, thereby arguing the inherent limitations of the numerical-threshold detection paradigm.
- We propose SIEVE, a lightweight Significant SDC detection framework. With a shared layer-aware predictor, SIEVE learns the nominal transformation of representations between adjacent layers from clean traces, and constructs a Discrepancy Profile from the deviation between predicted and actual representations as the detection signal for Significant SDCs, which a lightweight classifier then judges.
- We conduct a unified evaluation on three VLMs and three VQA datasets. Under the best configuration, SIEVE achieves a macro-average F1 of 93.78%, a recall of 91.49%, and an

FPR of 0.107% on the Final Test, improving over same-configuration baselines by up to 11.38 percentage points.

2 DESIGN MOTIVATION

2.1 DETECTION TARGET: SIGNIFICANT SDC

Following the definition in §1, an execution is labeled as a *Significant SDC* when its faulty output differs from the fault-free output and the semantic severity assessment judges it to contain severe factual errors or a deviation in the key conclusion. Semantic severity is evaluated by an LLM-as-a-Judge with a fixed rubric, and all samples are manually reviewed to ensure label reliability. As shown in Figure 2, among SDCs whose injected scalar remains finite before and after the fault and that carry a valid severity label, Significant SDCs account for 8.09%–92.09% of all SDCs, staying below 25% for most tasks; this indicates that the semantic impact of most SDCs is limited, and detection should focus on the subset with severe consequences rather than all numerical errors.

2.2 LIMITATIONS OF NUMERICAL ANOMALY DETECTION

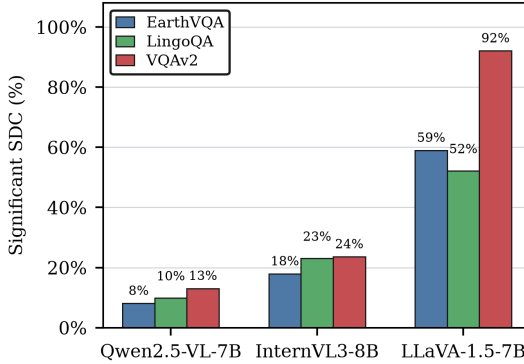


Figure 2: Fraction of Significant SDCs among all SDCs. Only SDCs whose injected scalar is finite before and after the fault and that carry a valid severity label are counted; the higher share for LLaVA stems from its short (2–3 token) answers.

Existing soft error detection methods such as Ranger and Dr.DNA typically judge fault impact by whether activations exceed normal ranges or deviate from normal distributions, implicitly assuming that a larger numerical deviation corresponds to a more severe impact. However, in VLMs, numerical deviation and output semantic damage are correlated yet not equivalent. Figure 3 analyzes only faults whose injected scalar remains finite before and after the fault, and bins them by numerical deviation magnitude into three intervals: Significant SDCs already exist in the low-deviation interval, while in the high-deviation interval most faults cause no significant semantic damage. Detecting with a fixed threshold (e.g., 1) would miss 1.37% of the Significant SDCs and raise unnecessary alarms for 24.1% of the non-significant SDCs. Therefore, Significant SDC detection cannot rely on the magnitude of nu-

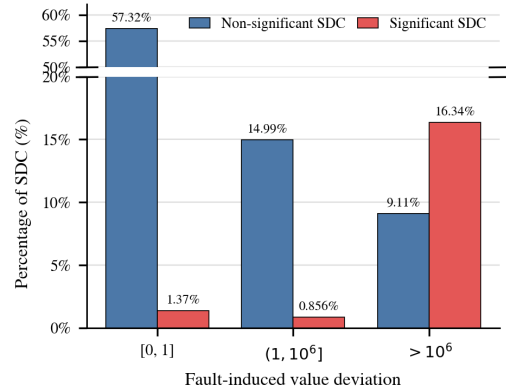


Figure 3: Distribution of SDCs and Significant SDCs across numerical deviation bins for finite-value faults.

merical anomalies itself, but must characterize how faults affect the inter-layer transformation of the model’s internal representations.

3 SIEVE DESIGN

3.1 OVERVIEW

The overall framework of SIEVE consists of three stages, as illustrated in Figure 4.

Stage 1: Nominal Inter-Layer Predictor. We first train the Nominal Inter-Layer Predictor on clean execution traces to learn the mapping between the projected attention features of adjacent monitored layers. At runtime, this predictor outputs the nominal reference feature of the target layer, providing a per-step baseline for subsequent discrepancy computation.

Stage 2: Significant SDC Detector. Under fault injection configurations, the Nominal Inter-Layer Predictor trained in Stage I is employed to predict the normal intermediate features at each monitored layer, which are then paired with the actual features produced during faulty execution to form predicted–actual feature pairs. For each feature pair, we extract both statistical and geometric discrepancies to construct a Discrepancy Profile. Combined with semantic labels generated by an LLM Judge, these representations are used to train a lightweight XGBoost detector capable of identifying SDCs that significantly impact the semantic outputs of the model.

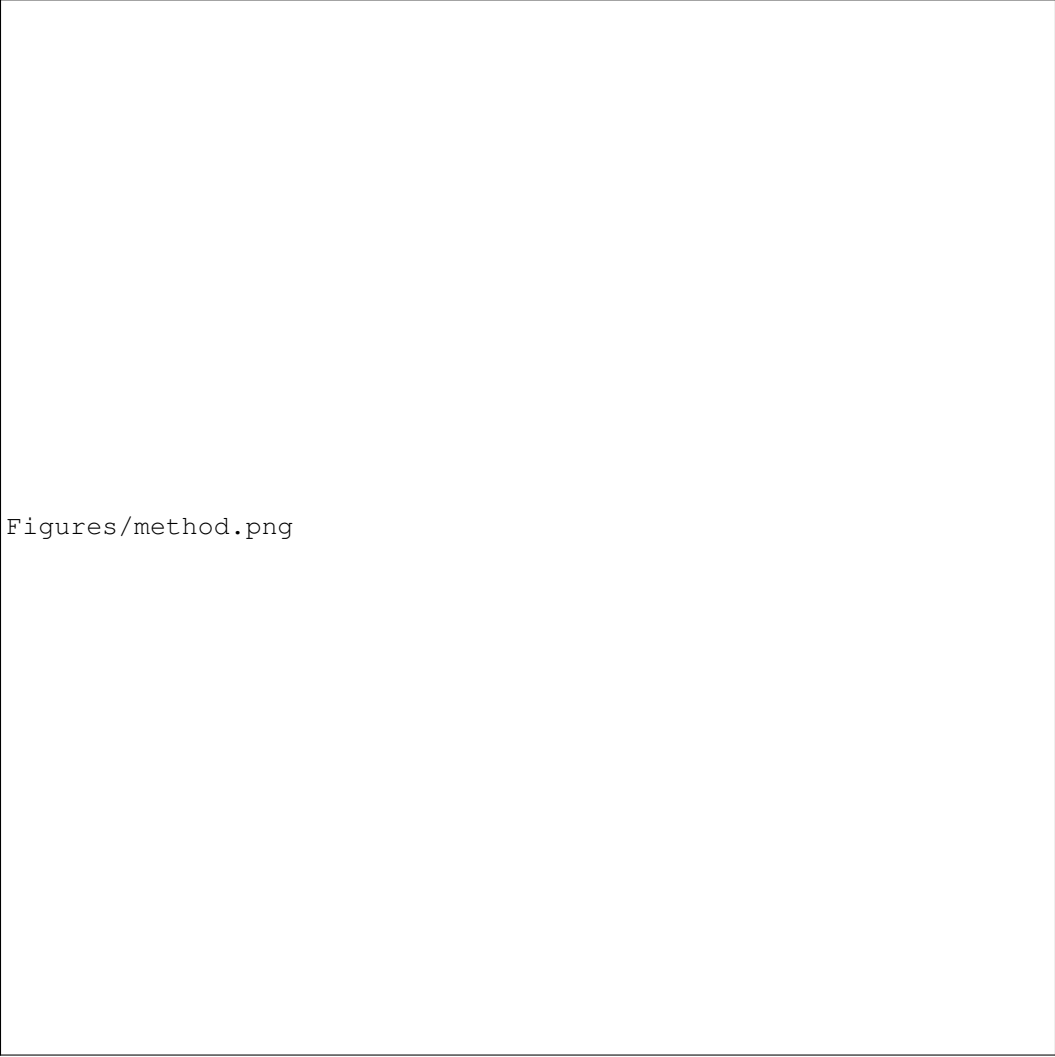
Stage 3: Runtime Deployment. During actual model deployment, only the trained Nominal Inter-Layer Predictor and the XGBoost detector need to be executed online to perform real-time Significant SDC detection based on runtime intermediate features. Since the entire detection process requires neither additional fault-free inference nor substantial computational complexity, it effectively reduces runtime overhead while maintaining detection capability.

3.2 NOMINAL INTER-LAYER PREDICTOR

3.2.1 FEATURE COLLECTION

The primary task of runtime detection is to distinguish feature discrepancies between SDC and Non-SDC executions. To this end, we leverage the Hook mechanism provided by PyTorch to collect, during clean execution, the output of the attention output projection (`o_proj`) of each monitored Decoder layer; for each decoding step we take the last-token vector, which is exactly the attention sub-layer contribution added to the residual stream. For every monitored adjacent layer pair $(\ell, \ell+1)$, the projected feature of layer ℓ serves as the predictor’s input and that of layer $\ell+1$ acts as the supervision signal, enabling the model to learn the nominal inter-layer transformation of attention features. However, directly training on raw Decoder features poses significant engineering challenges. Taking Qwen2.5-VL-7B as an example, its hidden dimension reaches 3,584. Storing all intermediate features not only consumes substantial memory, but such high-dimensional inputs also significantly inflate the parameter count of the predictor, degrade training efficiency, and increase the difficulty of learning the nominal feature mappings.

To address the challenges posed by high-dimensional features, we apply dimensionality reduction to the raw features after constructing the Feature Pairs. Our design is inspired by the Johnson–Lindenstrauss (JL) Lemma[?], which states that a set of high-dimensional vectors can be projected into a low-dimensional space whose dimension grows logarithmically with the sample size, while preserving pairwise distances within a small error bound. In other words, high-dimensional features need not be fully retained to maintain their geometric structure and similarity relationships. Guided by this theoretical foundation, we employ Column-Orthogonal Projection to map the high-dimensional Decoder features into a 64-dimensional space. The visualization and correlation analysis of features before and after projection are illustrated in Figure 5, where the Pearson correlation coefficient reaches 0.909. Compared to directly learning high-dimensional representations, this approach significantly reduces the storage overhead of intermediate features and effectively decreases the parameter count of the Predictor, thereby substantially improving training efficiency.



Figures/method.png

Figure 4: Overview of SIEVE. Per-layer attention outputs are projected to a low-dimensional space, a Nominal Inter-Layer Predictor models the nominal mapping between adjacent layers, and the resulting Discrepancy Profile feeds an XGBoost detector to flag Significant SDCs.

3.2.2 PREDICTOR TRAINING

To enable the predictor to accurately reconstruct the nominal intermediate features observed during clean execution, we perform targeted training on the Nominal Inter-Layer Predictor. Since intermediate features must maintain both numerical accuracy and semantic directionality, supervising with Mean Squared Error (MSE) alone tends to bias the model toward element-wise accuracy at the expense of overall semantic consistency in the feature space. Therefore, we further introduce a Cosine Similarity constraint to enforce directional alignment between predicted and ground-truth features in the representation space. Specifically, let $\hat{\mathbf{Y}}$ denote the predictor’s output and $\mathbf{Y}$ the corresponding observed clean-execution feature. The training loss is defined as:

$$L = L_{\text{mse}} + \lambda L_{\text{cos}},$$

where λ balances the two loss terms and is set to 0.1 in this work. The MSE term ensures numerical fidelity at the element level, while the cosine loss preserves directional consistency in the low-dimensional representation space, yielding more stable and semantically coherent predictions. After training, the predictor performs only forward inference at runtime to produce nominal feature estimates, which serve as the reference for the subsequent Discrepancy Profile.

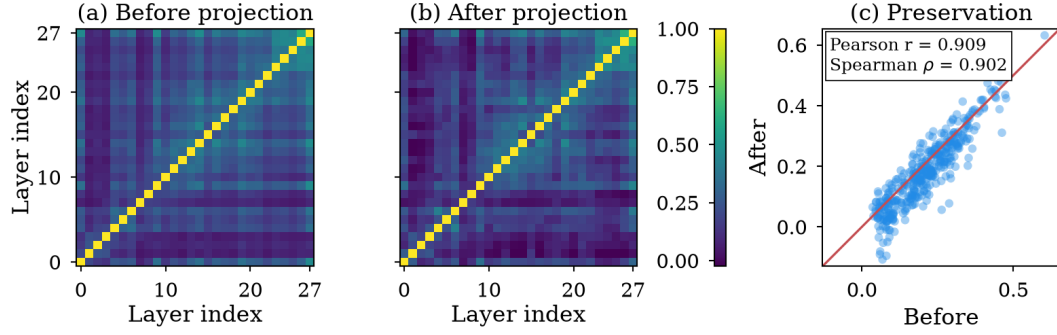


Figure 5: Feature structure preserved by the 64-dimensional column-orthogonal projection (Qwen2.5-VL-7B, LingoQA): pairwise similarity before vs. after projection.

3.3 SIGNIFICANT SDC DETECTOR TRAINING

3.3.1 DISCREPANCY PROFILE

Although the Nominal Inter-Layer Predictor is capable of predicting intermediate features under normal execution, the raw discrepancy between predicted and actual values cannot be directly used as the criterion for identifying Significant SDCs. On one hand, the predictor itself inevitably introduces inherent prediction errors. On the other hand, the impact of soft errors on intermediate features is diverse, manifesting as global numerical shifts, distribution fluctuations, or semantic direction deviations. Therefore, relying on a single metric is insufficient to comprehensively characterize the fault’s impact.

To address this, we design a Discrepancy Profile that extracts difference features between predicted and actual features from two complementary perspectives:

(1) Statistical Discrepancy: This captures overall changes in the feature distribution. Specifically, we compute the *signed* differences in mean and standard deviation between the predicted and actual features. The signed mean difference reflects global numerical shifts together with their direction, while the signed standard deviation difference captures changes in the dispersion of the feature distribution.

(2) Geometric Discrepancy: This reflects structural changes in the low-dimensional representation space. Specifically, we compute the cosine similarity and the Euclidean (L2) distance between the predicted and actual features. The cosine similarity measures directional consistency in the semantic representation space, while the L2 distance quantifies the absolute magnitude of feature deviation.

For each monitored layer pair, the four discrepancy metrics form a four-dimensional vector. Since soft errors exhibit different propagation behaviors across Transformer Decoder layers, we aggregate information over both depth and time: for every metric of every pair we compute its mean, maximum, and minimum over the first K decoding steps (the prefix aggregation window), and concatenate all statistics in a fixed order. With six adjacent layer pairs, four metrics, and three temporal statistics, the resulting Discrepancy Profile has a fixed dimensionality of $6 \times 4 \times 3 = 72$. This feature tuple integrates the propagation characteristics of errors across layer pairs, the complementary information from statistical and geometric perspectives, and the temporal evolution within the early decoding steps. It serves as the input to the Significant SDC Detector, enabling the model to learn the mapping between discrepancy patterns and error severity, thereby achieving accurate identification of Significant SDCs.

3.3.2 DETECTOR DATASET CONSTRUCTION

To train a detector capable of identifying Significant SDCs, we construct a dedicated training dataset for the Significant SDC Detector. The dataset construction involves two core components: fault injection with discrepancy feature extraction, and semantic label generation via an LLM Judge.

Fault Injection and Discrepancy Feature Extraction. Under the fault injection environment, we conduct extensive fault injection experiments on the target model, covering diverse injection sites. For each faulty execution, we extract the Discrepancy Profile between the predicted and actual features from all monitored layers, forming one training sample for the detector. Since each sample consists solely of low-dimensional statistical and geometric metrics extracted from multiple monitored layers, it is characterized by low dimensionality, minimal storage overhead, and ease of training.

Semantic Annotation via LLM Judge. After obtaining the discrepancy features, each sample must be assigned a corresponding supervision label. Since the output of VQA tasks is natural language text, merely checking whether the model’s prediction is correct is insufficient to accurately assess the actual impact of soft errors. Moreover, existing automatic evaluation metrics (e.g., BLEU, METEOR) primarily measure lexical or syntactic similarity and fail to capture the severity of errors at the semantic level. To this end, we employ an LLM Judge to perform automatic semantic annotation on faulty samples. Specifically, for each answer produced by a faulty execution, the LLM Judge jointly considers the ground-truth answer, the model’s fault-free output, and the faulty output to evaluate semantic consistency, determining whether the fault causes significant semantic deviation in the model’s output. Ultimately, each sample is assigned a binary label: Significant SDC or Slight/Non-SDC, serving as the supervision signal for the Detector.

3.3.3 SIGNIFICANT SDC DETECTOR

With the semantically labeled training data in hand, we adopt XGBoost as the Significant SDC classifier. Since the Discrepancy Profile is low-dimensional, XGBoost can efficiently model non-linear relationships among features in such a compact feature space. More importantly, compared to deep neural networks, XGBoost offers fast inference, a lightweight model footprint, and negligible runtime overhead, making it highly suitable for the stringent requirements of online runtime detection. After training, the final Significant SDC Detector is obtained. At runtime, the detector directly receives the Discrepancy Profile and rapidly determines whether the current sample constitutes a Significant SDC, providing a reliable basis for subsequent error recovery or fault tolerance mechanisms.

Through the above process, SIEVE transforms the complex problem of semantic error detection into a supervised classification task based on low-dimensional feature representations, significantly reducing runtime computational overhead while maintaining detection accuracy.

3.4 ONLINE DETECTION WORKFLOW

During online deployment, SIEVE performs no training or parameter updates; instead, it integrates with the original VLM in a bypass manner, executing synchronously with the inference process. As illustrated in Figure 4, SIEVE retrieves the output features of pre-selected Decoder layers via PyTorch Forward Hooks. These hooks solely read intermediate features and return the original outputs, ensuring that the model’s activations, logits, and generation logic remain unmodified. In each decoding step, SIEVE collects the `o_proj` output of the layers involved in the six selected adjacent pairs and maps them into a 64-dimensional space using a fixed column-orthogonal projection matrix. All monitored layers share this single projection matrix, which reduces subsequent computational overhead while preserving the relative geometric relationships among features across different layers. Subsequently, for each monitored layer pair, SIEVE feeds the low-dimensional features of the source layer into the Nominal Inter-Layer Predictor to estimate the nominal features of the target layer. These predictions are then compared with the target layer’s actual features to construct a Discrepancy Profile that captures the semantic deviations induced by inter-layer fault propagation.

To enable early detection and control runtime overhead, SIEVE restricts temporal aggregation to the intermediate features from the first K decoding steps. For each discrepancy metric within every layer pair, SIEVE computes the mean, maximum, and minimum values across these K steps, concatenating all statistical results in a fixed order to form a 72-dimensional Discrepancy Profile. In our current deployment, K is set to 2; if an EOS token is generated before K steps are collected, SIEVE utilizes all available steps for feature aggregation, ensuring robust detection even for short outputs. The trade-off between detection latency and information completeness regarding K , along with its ablation study, is detailed in the experimental section. Finally, the 72-dimensional

Table 1: Qualitative comparison of detection mechanisms

Method	Signal	Semantic	Finite-val	Offline	VLM-ready
Ranger	Numerical range	No	No	Yes	No
Dr.DNA	Statistical deviation	No	No	Yes	No
NaN/Inf check	Non-finite values	No	Yes	No	Yes
SIEVE (Ours)	NaN/Inf + Semantic	Yes	Yes	Yes	Yes

Table 2: Quantitative baselines for comparison

Baseline	Detection mechanism
NaN/Inf Only	Detect non-finite values only
NaN/Inf + Range-based	Layer-wise numerical range detection (inspired by Ranger)
NaN/Inf + Statistics-based	Statistical numerical deviation detection (inspired by Dr.DNA)
SIEVE (Ours)	NaN/Inf + semantic detector

Discrepancy Profile is fed into the pre-trained XGBoost Detector, which outputs the probability of a Significant SDC and the final binary decision. If a Significant SDC is identified, subsequent recovery or fault-tolerance mechanisms are triggered; otherwise, normal generation proceeds.

The additional computation during the online phase primarily consists of the low-dimensional projection for six layers, batch feature prediction for six layer pairs, lightweight statistical calculations, and a single XGBoost inference. Since SIEVE eliminates the need to re-execute VLM inference, maintain model replicas, or store online Golden Features, its computational and memory overheads are significantly lower than those of redundant execution-based detection methods. Detailed metrics regarding end-to-end latency, throughput, and memory footprint are reported in the experimental section.

4 EXPERIMENTS

4.1 COMPARISON WITH EXISTING METHODS

Qualitative Mechanism Comparison. Table 1 presents a qualitative comparison of detection mechanisms between SIEVE and existing approaches. We compare along five dimensions: detection signal, semantic awareness, coverage of finite-value errors, requirement for offline profiling, and applicability to VLMs.

Quantitative Baselines. Since Ranger and Dr.DNA do not provide public implementations and omit details required for faithful reproduction, we implement two mechanism-matched baselines following their core detection principles. We do not claim these implementations reproduce the original systems. Table 2 summarizes the quantitative baselines.

To ensure a fair comparison, we adopt the following protocol: (1) All methods share identical fault injection records, models, and datasets. (2) All methods share the same NaN/Inf fast detection to prevent SIEVE from gaining an additional advantage. (3) Thresholds are tuned exclusively on the validation set; the test set remains fixed. (4) Each method is allocated the same hyperparameter tuning budget. (5) All methods employ the same recovery strategy (i.e., re-executing the current sample upon detection). (6) We report mean and standard deviation over multiple random trials. We further report two overhead metrics: *Recovery Rate* (the proportion of samples triggering recovery) and *End-to-End Overhead* (the combined cost of detection and recovery), as the per-recovery cost is nearly identical under a unified recovery strategy.

Table 3: Final validation performance of the Nominal Inter-Layer Predictors across VLMs and datasets.

Model	EarthVQA		LingoQA		VQAV2	
	MSE	CosSim	MSE	CosSim	MSE	CosSim
Qwen2.5-VL-7B	0.1128	0.8194	0.1637	0.7436	0.2139	0.6414
InternVL3-8B	0.0919	0.7593	0.1191	0.6802	0.1292	0.5899
LLaVA-1.5-7B	0.0035	0.9470	0.0067	0.8729	0.0144	0.7636

4.2 NOMINAL INTER-LAYER PREDICTOR

4.2.1 MODEL TRAINING

We evaluate the prediction quality of the Nominal Inter-Layer Predictor across three vision-language models (VLMs) and three datasets. All metrics are computed on held-out test sets that were not used during training. As shown in Table 3, the predictor learns stable nominal inter-layer feature relationships across different models and datasets. All three models achieve positive cosine similarity between predicted and observed features on every dataset, though the prediction difficulty varies across datasets for a given model.

It is important to note that different VLMs exhibit distinct activation distributions, visual encoding pathways, and language backbones. Therefore, the absolute MSE values should not be directly used to rank predictive capability across models. The key takeaway is that the predictor provides directionally consistent approximations of the true nominal features across all nine model–dataset combinations, thereby offering stable inputs for the subsequent Significant SDC detection based on prediction residuals.

4.2.2 SDC FEATURE VISUALIZATION

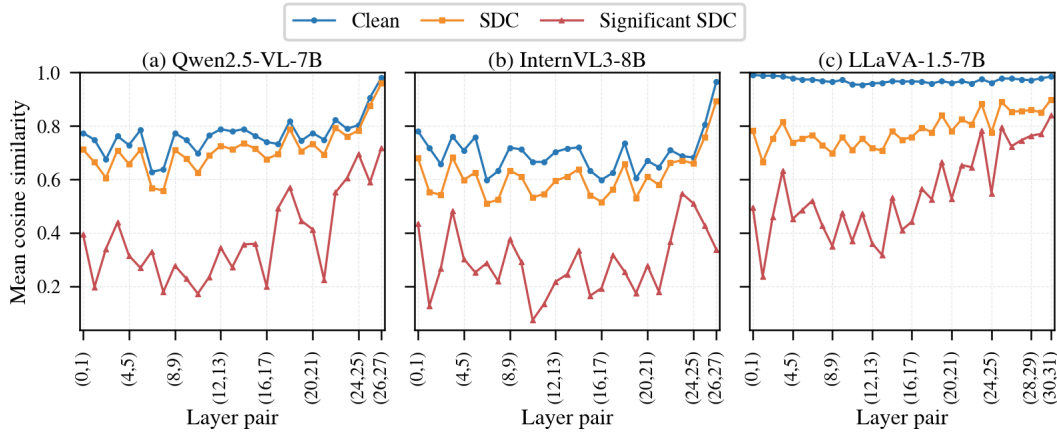


Figure 6: Inter-layer cosine similarity (mean over decoding steps and samples) for Non-SDC, SDC, and Significant SDC executions of three models on LingoQA.

Figure 6 illustrates the inter-layer cosine similarity between SDC and Non-SDC executions across three models on LingoQA. For each sample, we first average the cosine similarity of a given layer pair over multiple decoding steps, and then average across samples of the same category. Here, Non-SDC refers to executions where fault injection does not alter the output, SDC encompasses all executions that produce output changes, and Significant SDC denotes the subset of SDC cases that cause substantial semantic impact.

The three models exhibit a consistent trend: Non-SDC executions generally achieve the highest inter-layer cosine similarity, indicating that their actual features align well with the normal feature evolution predicted by the Nominal Inter-Layer Predictor. When faults lead to SDC, the cosine

Table 4: Detection performance (%) for Significant SDC across VLMs and datasets without filtering NaN/Inf samples

Model	LingoQA			EarthVQA			VQAv2		
	P	R	F1	P	R	F1	P	R	F1
Qwen2.5-VL-7B	94.98	89.66	92.24	90.48	90.87	90.67	99.07	77.82	87.17
LLaVA-1.5-7B	94.48	87.26	90.73	97.48	87.57	92.26	92.41	91.78	92.10
InternVL3-8B	99.58	95.60	97.55	99.32	97.10	98.19	98.88	93.04	95.87

Table 5: Detection performance (%) for Significant SDC across VLMs and datasets with NaN/Inf samples filtered out

Model	LingoQA			EarthVQA			VQAv2		
	P	R	F1	P	R	F1	P	R	F1
Qwen2.5-VL-7B	92.90	85.71	89.16	87.71	88.20	87.96	98.62	70.10	81.95
LLaVA-1.5-7B	97.62	67.21	79.61	91.49	66.15	76.79	93.94	72.09	81.58
InternVL3-8B	99.48	94.59	96.98	99.15	96.44	97.78	98.61	91.49	94.92

similarity decreases overall; the decline is even more pronounced for Significant SDC, forming clear separations from both Non-SDC and general SDC across most layer pairs. This suggests that faults causing substantial semantic impact more severely disrupt normal cross-layer feature relationships.

Moreover, the inter-class gaps vary noticeably across different layer pairs, indicating that fault propagation is layer-dependent. Therefore, monitoring a single layer or applying a uniform numerical threshold is insufficient for reliably identifying Significant SDC. SIEVE aggregates prediction residuals from multiple layer pairs, leveraging cross-layer anomaly patterns to distinguish Non-SDC, general SDC, and Significant SDC. These results validate that the residuals produced by the Nominal Inter-Layer Predictor effectively characterize the semantic impact of faults.

4.3 SDC DETECTOR

4.3.1 DETECTION PERFORMANCE

Table 4 and Table 5 report the detection performance for Significant SDCs across different VLMs and datasets, with and without filtering NaN/Inf samples, respectively. Overall, SIEVE achieves robust detection across all settings, with F1 scores mostly exceeding 87% and InternVL3-8B consistently delivering top performance (>95%). Comparing the two tables reveals that filtering out NaN/Inf samples degrades the performance to varying degrees, exposing distinct model dependencies. Specifically, InternVL3-8B remains highly robust with negligible F1 drops, demonstrating its ability to capture Significant SDCs through subtle feature deviations. In contrast, LLaVA-1.5-7B and Qwen2.5-VL-7B experience more substantial performance drops (especially in recall) when numerical artifacts are removed, suggesting that their detection partially relies on extreme NaN/Inf values as strong signals. Despite these variations, the consistently high precision across all configurations ensures a low false alarm rate critical for online deployment, validating that SIEVE effectively identifies Significant SDCs even without relying on fatal numerical errors.

4.3.2 DETECTION OVERHEAD

We further evaluate the online deployment overhead of SIEVE on the NVIDIA H20 GPU. The online implementation monitors only six layers and completes projection, fault-free feature prediction, 72-dimensional difference-feature aggregation, and XGBoost inference after the first two decoding steps. To mitigate biases introduced by execution order and GPU thermal states, we repeat measurements in both forward and reverse orders, yielding a total of 200 end-to-end observations for each model and mode. The end-to-end latency overhead of full SIEVE on Qwen2.5-VL-7B,

Table 6: Online deployment overhead of SIEVE on NVIDIA H20 GPUs.

Model	Baseline (ms)	SIEVE (ms)	Latency overhead	Throughput change
Qwen2.5-VL-7B	538.04	544.06	+1.12%	-0.91%
InternVL3-8B	671.75	676.66	+0.73%	-0.72%
LLaVA-1.5-7B	252.50	255.88	+1.34%	-1.32%

InternVL3-8B, and LLaVA-1.5-7B is 1.12%, 0.73%, and 1.34%, respectively, with an average of merely 1.06% across the three models; the corresponding average throughput degradation is 0.98%. SIEVE requires only an extra 44.33–148.85ms on average to finish judgment after prefill. For short outputs that produce EOS early, detection is performed immediately using existing steps, maintaining 100% detection coverage.

4.4 ABLATION STUDIES

4.4.1 EFFECT OF MONITORED LAYER SELECTION

To analyze the contribution of each monitored layer pair to Significant SDC detection, we conduct a Leave-One-Layer-Out ablation study across three models. In each trial, one layer pair is removed while the remaining five are retained (yielding 60-dimensional features). Data splitting, the detector architecture, and hyperparameters remain unchanged across all trials. To eliminate interference from varying NaN distributions, all configurations are evaluated on the exact same non-all-NaN validation set defined by the full six-layer-pair setup. Table 7 summarizes the F1 scores across the three models and three datasets.

The experimental results reveal two key findings. First, the terminal layer pairs provide the most stable cross-model discriminative information. Averaged across the three models, removing L25 results in the largest drop in average F1 (0.98 percentage points), followed by L27 (0.68 percentage points). This indicates that cross-layer residual features near the model’s output end are crucial for discriminating Significant SDCs. Second, the contribution of early layer pairs is highly model-dependent. Removing L7 causes a 0.40 percentage point drop in F1 on InternVL3-8B, but surprisingly improves F1 by 1.12 percentage points on Qwen2.5-VL-7B and shows negligible change (+0.03 percentage points) on LLaVA-1.5-7B. This suggests that fault propagation paths are inconsistent across different models. Overall, the ablation results validate the necessity of multi-layer joint monitoring and demonstrate that the selection of monitoring positions is more critical than merely increasing the number of layers.

4.4.2 EFFECT OF AGGREGATION STEP SIZE

We will revise the paper’s positioning to prioritize online deployment: we select $K = 2$, whose average performance is close to that of the full trajectory while requiring the shortest detection waiting time. Accordingly, we will update K^* in the existing line plot from 12 to 2 to avoid inconsistency between the figure and the textual conclusion. The paper will explicitly clarify that what is reduced here is the observation window/detection latency, which cannot be directly equated with a reduction in end-to-end runtime.

The current script defines K^* as the smallest K for which all three models remain within 1 percentage point, and therefore automatically yields 12. This is inconsistent with the system objective currently adopted in the paper. We will change it to an explicit deployment choice, ‘-selected-k’, with a default value of 2. In this way,

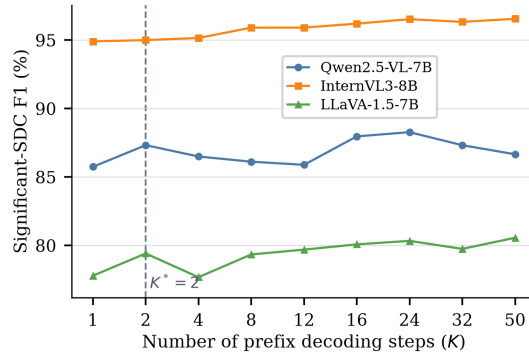


Figure 7: online step ablation

Table 7: Leave-One-Layer-Out ablation across three VLMs (F1, %).

Config	Layers	EarthVQA	LingoQA	VQAv2	Avg.	Δ Avg.
<i>Qwen2.5-VL-7B</i>						
Full	6	87.96	90.10	81.95	86.67	–
w/o L7	5	90.21	90.73	82.42	87.79	+1.12
w/o L23	5	88.89	89.46	81.48	86.61	–0.06
w/o L24	5	88.95	90.79	82.08	87.28	+0.61
w/o L25	5	88.27	89.10	81.61	86.33	–0.34
w/o L26	5	89.74	89.39	81.84	86.99	+0.32
w/o L27	5	87.47	91.08	81.74	86.76	+0.10
<i>InternVL3-8B</i>						
Full	6	97.78	96.99	94.92	96.56	–
w/o L7	5	96.57	96.99	94.92	96.16	–0.40
w/o L23	5	97.51	96.85	94.79	96.38	–0.18
w/o L24	5	97.51	96.86	95.05	96.47	–0.09
w/o L25	5	96.32	96.37	94.92	95.87	–0.69
w/o L26	5	97.38	96.98	95.29	96.55	–0.01
w/o L27	5	96.85	96.73	94.18	95.92	–0.64
<i>LLaVA-1.5-7B</i>						
Full	6	76.79	83.33	81.58	80.57	–
w/o L7	5	76.11	84.11	81.58	80.60	+0.03
w/o L23	5	76.11	83.33	79.49	79.64	–0.92
w/o L24	5	74.78	84.11	79.49	79.46	–1.11
w/o L25	5	75.44	83.02	77.50	78.65	–1.91
w/o L26	5	77.48	83.33	79.49	80.10	–0.47
w/o L27	5	75.44	83.33	78.48	79.08	–1.48

Table 8: Significant SDC detection performance under different prefix aggregation windows (F1, %).

K	Qwen	InternVL	LLaVA	Avg. F1	Δ vs. $K = 50$	Avg. steps
1	85.75	94.91	77.80	86.15	–1.78	1.00
2	87.33	95.01	79.42	87.25	–0.68	1.88
4	86.50	95.16	77.68	86.45	–1.48	3.47
8	86.12	95.92	79.35	87.13	–0.80	6.48
12	85.90	95.92	79.70	87.17	–0.76	9.25
16	87.97	96.21	80.08	88.09	+0.16	11.64
24	88.28	96.53	80.34	88.38	+0.45	14.74
32	87.33	96.34	79.75	87.81	–0.13	15.93

the figure will only reflect the strategy selected in the paper and will no longer imply a different optimization criterion.

We aggregate detection features using the first K decoding steps and independently train XGBoost for each K . All configurations use the same data splits and a fixed non-all-NaN validation set.

To analyze the detection latency of SIEVE in online deployment scenarios, we restrict the feature aggregation window to the first K decoding steps of the generated sequence, with $K \in \{1, 2, 4, 8, 12, 16, 24, 32, 50\}$. For outputs with fewer than K steps, we use all available features up to EOS. For each window, we independently train the detector and use the same data splits and fixed validation sample set, thereby excluding the effect of sample changes on the results.

The results show that fault-induced abnormal inter-layer representations can be effectively captured early in generation. When using only the first two decoding steps, SIEVE achieves an average F1 of 87.25% across the three models and three datasets, which is only 0.68 percentage points lower than the 87.93% obtained with full trajectories. Meanwhile, the actual average observation length decreases from 17.05 to 1.88 steps, a reduction of approximately 89.0%. Although $K = 24$ achieves the highest average F1, its observation window is about 7.8 times larger than that of $K = 2$, while improving performance by only 1.13 percentage points. Therefore, we choose $K = 2$ as the default

online aggregation window, trading a small accuracy loss for a substantial reduction in detection waiting time.

5 OTHERS

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

In this work, we used generative AI tools for [tasks with required disclosure]. We have not used generative AI tools for [other tasks with required disclosure], and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. [Elaborate. For example, “we checked LLM-generated research ideas for potential plagiarism through a manual literature survey”, “LLM-generated code was verified and tested for correctness by 2 authors”, etc.]. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

See the ICLR 2027 AI Policy for Authors for more details. This statement should not be more than 1 page.

ETHICS STATEMENT

(This section is **recommended** and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

REPRODUCIBILITY STATEMENT

(This section is **recommended** and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

AUTHOR CONTRIBUTIONS

If you’d like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors.

ACKNOWLEDGMENTS

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

REFERENCES

- Abdel Rahman Alsabbagh, Tariq Mansour, Mohammad Al-Kharabsheh, Abdel Salam Ebdah, Roa'a Al-Emaryeen, Sara Al-Nahhas, Waleed Mahafza, and Omar Al-Kadi. Minimedgpt: efficient large vision-language model for medical visual question answering. *Pattern Recognition Letters*, 189: 8–16, 2025.
- Yi He, Mike Hutton, Steven Chan, Robert De Gruijl, Rama Govindaraju, Nishant Patil, and Yan-jing Li. Understanding and mitigating hardware failures in deep learning training systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pp. 1–16, 2023.
- Chonghao Sima, Katrin Renz, Kashyap Chitta, Li Chen, Hanxue Zhang, Chengen Xie, Jens Beißwenger, Ping Luo, Andreas Geiger, and Hongyang Li. Drivelm: Driving with graph visual question answering. In *European conference on computer vision*, pp. 256–274. Springer, 2024.
- Jingweijia Tan, Xurui Li, An Zhong, Kaige Yan, Xiaohui Wei, and Guanpeng Li. Gerem: Fast and precise error resilience assessment for gpu microarchitectures. *IEEE transactions on parallel and distributed systems*, 2025.
- Xiaoyu Tian, Junru Gu, Bailin Li, Yicheng Liu, Yang Wang, Zhiyong Zhao, Kun Zhan, Peng Jia, Xianpeng Lang, and Hang Zhao. Drivevlm: The convergence of autonomous driving and large vision-language models. *arXiv preprint arXiv:2402.12289*, 2024.
- Junjue Wang, Weihao Xuan, Heli Qi, Zhihao Liu, Kunyi Liu, Yuhan Wu, Hongruixuan Chen, Jian Song, Junshi Xia, Zhuo Zheng, et al. Disasterm3: A remote sensing vision-language dataset for disaster damage assessment and response. *arXiv preprint arXiv:2505.21089*, 2025.
- Shaobu Wang, Guangyan Zhang, Junyu Wei, Yang Wang, Jiesheng Wu, and Qingchao Luo. Understanding silent data corruptions in a large production cpu population. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 216–230, 2023.
- Maolin Wei, Wanzhou Liu, and Eshed Ohn-Bar. Passing the driving knowledge test. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 8395–8406, 2025.

A APPENDIX

You may include other additional sections here.